

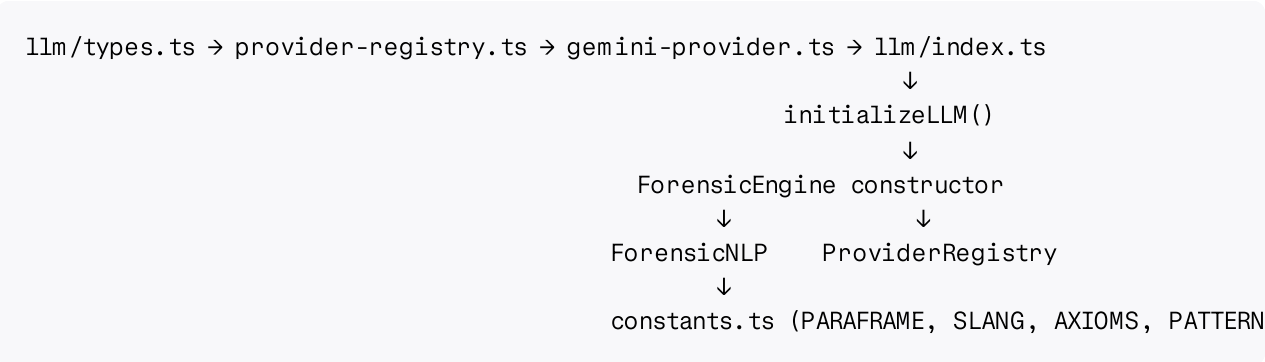
Operační Dependency Matice — LEX FORENSICA v8.0

Real-Time Vývojová Koncepce

[1] SINGLETON DEPENDENCY CHAIN (Runtime Initialization Order)



Kritická Cesta Inicializace:



BLOCKER: Pokud `initializeLLM()` neproběhne před prvním `analyzeDeep()`, `resolvePhase()` vyhodí "provider not registered" error. Toto je jednosměrná závislost bez fallbacku.

[2] INSTRUKČNÍ DEPENDENCY CHAIN (Prompt → Type → Validation)

Pipeline Fáze → Placeholder → TypeScript Interface

PIPELINE	PROMPT PLACEHOLDERS	MAPPED TypeScript INTERFACE
MIND1	{{INPUT_A}} {{INPUT_B}} {{LANGUAGE}} {{INSTITUTION_TYPE}} OUTPUT →	AuditInput.inputA: string AuditInput.inputB: string AuditInput.language: 'cs' 'en' 'de' AuditInput.institutionType: enum NormalizedEventFrame[] 11 fields per frame JSON.parse() from LLM text
[BRIDGE] hashFact()	– (programmatic)	FactCheckpoint[] SHA-256 locked source: STRONG frames only
DEEP_1	{{FRAMES}} {{IMMUTABLE_ANCHORS}} {{INSTITUTION_TYPE}} {{LANGUAGE}} {{TIER}} OUTPUT →	NormalizedEventFrame[] (MIND1 out) FactCheckpoint[] (bridge out) AuditInput.institutionType AuditInput.language SubscriptionTier enum AuditResponse 18 top-level fields ~200 nested fields total
LOOP_CYCLE verifyIntegr.	– (programmatic) audit + inputA/B	LoopCycleViolation[] 8 check categories max 2 re-runs on CRITICAL
DEFENSE	{{AUDIT_RESPONSE}} {{FRAMES}} {{CHECKPOINTS}} OUTPUT →	AuditResponse (DEEP_1 out) NormalizedEventFrame[] (MIND1 out) FactCheckpoint[] (bridge out) DefenseSynthesis 6 fields → audit.defenseSynthesis = def

Klíčová Závislost: Prompt Placeholder ↔ Runtime Substitution

prompts.ts

MIND1_PROMPT

DEEP1_PROMPT

...

⚠ DRIFT RISK: Placeholder v promptu = {{FRAMES}}

Runtime substitute = {{FRAMES_JSON}}

Pokud se prompt změní a runtime ne (nebo naopak),

LLM dostane raw placeholder string jako vstup.

gemini.ts

extractFrames()

.replace('{{INPUT_A}}', inputA)

.replace('{{INPUT_B}}', inputB)

.replace('{{LANGUAGE}}', language)

.replace('{{INSTITUTION_TYPE}}', institutionType)

runDeepAnalysis()

.replace('{{FRAMES_JSON}}', JSON.stringify(frames))

.replace('{{CHECKPOINTS_JSON}}', ...)

NALEZENÝ POTENCIÁLNÍ BUG: DEEP1_PROMPT obsahuje {{FRAMES}} a {{IMMUTABLE_ANCHORS}}, ale runDeepAnalysis() substituuje {{FRAMES_JSON}} a {{CHECKPOINTS_JSON}}. Toto je buď:

- (a) záměrný duální klíč (prompt má oba), nebo
- (b) nesynchronizovaný placeholder — DEEP_1 prompt může obsahovat nesubstituované {{FRAMES}} vedle správně nahrazených {{FRAMES_JSON}}.

→ Verifikace nutná v gemini.ts:runDeepAnalysis().

[3] AXIOM → VALIDATION → LEGAL DEPENDENCY

constants.ts		nlp-core.ts		prompts.ts
AXIOM_DEFINITIONS[A1]	→	verifyIntegrity():		DEEP1_PROMPT:
.name		A1 check:		"A1 – FORENSIC SPOILIATION"
.description		documentationGaps.length > 0		(full description inline)
.legalBasis: "ECHR 3,13"		&& !A1 in axiomaticViolations		
		→ LoopCycleViolation(A1, HIGH)		
AXIOM_DEFINITIONS[A2]	→	A2 check:		"A2 – SEMANTIC NEUTRALIZATION"
.legalBasis: "ECHR 6"		complianceTerms hit in inputA		(full description inline)
		&& !A2 in discrepancyMatrix		
		→ LoopCycleViolation(A2, HIGH)		
AXIOM_DEFINITIONS[A3]	→	A3 check:		"A3 – IATROGENIC ATTRIBUTION"
.legalBasis: "ECHR 3, CRPD 25"		iatrogenicTerms hit in inputA		(full description inline)
		&& !A3 in axiomaticViolations		
		→ LoopCycleViolation(A3, HIGH)		
AXIOM_DEFINITIONS[A4]	→	A4 check:		"A4 – NON-FALSIFIABILITY"
.legalBasis: "ECHR 6(1)"		circularTerms hit		(full description inline)

```

                                && !falsifiabilityCheck
                                → LoopCycleViolation(A4, MED)

AXIOM_DEFINITIONS[A5]    →  A5 check:                                "A5 – STRUCTURAL BIAS"
    .legalBasis: "ECHR 6(1),    expertNameCounts > 1                (full description inline)
    CRPD 12"                && !A5 in axiomaticViolations
                                → LoopCycleViolation(A5, MED)

AXIOM_DEFINITIONS[A6]    →  A6 check:                                "A6 – JUDICIAL DELEGATION"
    .legalBasis: "ECHR 5(4),    judicialTerms hit                (full description inline)
    6(1)"                && !independentAssessment
                                → LoopCycleViolation(A6, HIGH)

– (no constants)          →  TRIPARTITE check:                    – (not in prompt)
                                dignity in primaryConcerns
                                && !dignity in immediateActions
                                → LoopCycleViolation(TRIP, CRIT)

– (no constants)          →  RULE_001 check:                    – (not in prompt)
                                inputB.trim().length < 50
                                → LoopCycleViolation(R001, CRIT)

```

Tři-Vrstvá Axiomová Závislost:

```

VRSTVA 1 (Definice):      constants.ts  → AXIOM_DEFINITIONS, INSTITUTION_PATTERNS
                                ↓
VRSTVA 2 (Instrukce):     prompts.ts    → A1-A6 inline v DEEP1_PROMPT
                                (duplikace! ne reference na constants)
                                ↓
VRSTVA 3 (Validace):      nlp-core.ts   → verifyIntegrity() term-matching
                                (závisí na getComplianceFramingTerms() atd.)

```

DRIFT RISK: Axiom definice existují na 3 místech nezávisle:

1. constants.ts → AXIOM_DEFINITIONS (strukturovaný objekt)
2. prompts.ts → DEEP1_PROMPT (inline text v promptu)
3. nlp-core.ts → hardcoded term lists v metodách

Pokud se změní axiom v jednom místě a ne v ostatních, systém bude mít **vnitřní nekonzistenci**, kterou LOOP_CYCLE nedetekuje, protože LOOP_CYCLE validuje output, ne zdrojový kód.

[4] INTERFACE PROPAGATION MAP — Kde AuditResponse teče

```

AuditResponse (types.ts, 317 lines)
|
|— CREATED BY:
|   |— gemini.ts: runDeepAnalysis()      → parsed from LLM JSON
|   |— gemini.ts: buildScaffoldAudit()   → fallback on parse failure
|
|— CONSUMED BY (runtime):
|   |— gemini.ts: runLoopCycleValidation(audit, ...) → nlp-core.ts

```

```

|   └─ gemini.ts: synthesizeDefense(audit, ...)      → DEFENSE_PROMPT
|   └─ gemini.ts: chatWithAssistant(..., auditContext) → ASSISTANT_PROMPT
|   └─ hooks/useAudit.ts: audit state                → React state
|
| └─ CONSUMED BY (UI components):
|   └─ Dashboard.tsx          → tabs, overall layout
|   └─ RiskGauge.tsx          → riskAssessment block
|   └─ DiscrepancyMatrix.tsx  → discrepancyMatrix[]
|   └─ LegalMatrix.tsx        → legalMatrix[]
|   └─ ChronologyTimeline.tsx → chronology[]
|   └─ DefenseDraft.tsx       → defenseSynthesis
|   └─ ExportPanel.tsx        → full audit (PDF/HTML gen)
|   └─ ChatAssistant.tsx      → context injection
|
| └─ STORED (encrypted):
|   └─ db.ts: saveAudit()      → encryptData(JSON.stringify(audit))
|                               → Firestore: StoredAudit

```

Critical Path: Pokud se změní structure `AuditResponse`, je potřeba aktualizovat 11 souborů simultánně. Žádný runtime type-check na LLM output (jen `JSON.parse()` + hope).

[5] BLOCKER ANALÝZA — Kritická Cesta Vývoje

BLOCKER-01: LLM Output ↔ TypeScript Interface Validate

STAV: ⚠ PARTIAL
 POPIS: DEEP_1 prompt definuje JSON schema v textu, ale runtime dělá jen `JSON.parse()` bez schema validation. Pokud LLM vynechá field nebo přidá neočekávaný typ, komponenta crashne.
 DOPAD: Jakýkoliv LLM model swap (nový Gemini, Claude, GPT) může rozbít celý pipeline bez varování.
 ŘEŠENÍ: Zod/Pydantic schema validace na LLM output boundary.
 EFFORT: 2 dny | PRIORITY: HIGH

BLOCKER-02: Axiom Triple-Definition Drift

STAV: ⚠ ACTIVE RISK
 POPIS: Axiomy A1-A6 definovány na 3 místech (constants, prompts, nlp-core) bez synchronizačního mechanismu.
 DOPAD: Přidání A7-A13 (plánováno v Lex Forensica skillu) vyžaduje ruční úpravu všech 3 souborů. Chybějící update = tichý drift.
 ŘEŠENÍ: Single source of truth v constants.ts → auto-generovat prompt sekce a validační logiku.
 EFFORT: 3-4 dny | PRIORITY: HIGH

BLOCKER-03: Placeholder Key Mismatch

STAV: ⚠ NEEDS VERIFICATION
POPIS: DEEP1_PROMPT používá {{FRAMES}} + {{IMMUTABLE_ANCHORS}},
gemini.ts substituuje {{FRAMES_JSON}} + {{CHECKPOINTS_JSON}}.
DOPAD: Pokud prompt obsahuje oba klíče, může zůstat nesubstituovaný
placeholder v LLM vstupu.
ŘEŠENÍ: Sjednotit placeholder konvenci. Přidat runtime check:
if (prompt.includes('{{')) throw new Error('Unresolved placeholder');
EFFORT: 0.5 dne | PRIORITY: CRITICAL

BLOCKER-04: Pipeline Hash Chain (Identified in Gemini Audit)

STAV: ✗ MISSING
POPIS: FactCheckpoint hashuje jednotlivé fakty, ale chybí chain-of-custody
hash na úrovni pipeline fáze (MIND1 output → DEEP_1 input).
DOPAD: Nelze kryptograficky prokázat, že DEEP_1 dostal přesně ten output,
který MIND1 vyprodukoval. Při retry/re-run se může payload změnit.
ŘEŠENÍ: SHA-256 hash na JSON.stringify(frames) před vstupem do DEEP_1,
přidat do audit.meta jako pipelineHashes.
EFFORT: 1 den | PRIORITY: MEDIUM

BLOCKER-05: LOOP_CYCLE Nepokrývá A7-A13

STAV: ✗ MISSING
POPIS: Lex Forensica skill definuje axiomy A1-A13, ale nlp-core.ts
implementuje verifyIntegrity() jen pro A1-A6 + TRIPARTITE + RULE_001.
DOPAD: Axiomy A7 (RETRO), A8 (DRIFT), A9 (CONSENT), A10 (RISK),
A11 (VOICE), A12 (SELF CERT), A13 (META) nemají LOOP_CYCLE validaci.
ŘEŠENÍ: Rozšířit OperationalAxiom enum a verifyIntegrity() o A7-A13.
EFFORT: 3-5 dní | PRIORITY: HIGH

BLOCKER-06: No Automated Test Suite

STAV: ✗ MISSING
POPIS: Žádné unit testy, integration testy, ani axiom compliance testy.
Vývoj závisí na manuální verifikaci.
DOPAD: Jakákoliv refaktORIZACE (type change, prompt update, axiom add)
nemá safety net. Regrese jsou tiché.
ŘEŠENÍ: Vitest/pytest test suite (viz Python integration plán).
EFFORT: 5-7 dní | PRIORITY: HIGH

[6] OPERAČNÍ SEKVENCE — Doporučený Vývojový Postup

FÁZE 0 (Okamžitě – 1 den):

- └─ FIX: Verifikovat placeholder mismatch (BLOCKER-03)
- └─ ADD: Unresolved placeholder guard v ForensicEngine
- └─ ADD: Pipeline hash na MIND1 output (BLOCKER-04)

FÁZE 1 (Týden 1 – Foundation):

- └─ REFACTOR: Single source axiom definition
 - | constants.ts → auto-gen prompt sections
 - | constants.ts → auto-gen verifyIntegrity() checks
- └─ ADD: Zod schema validace na LLM output boundary
- └─ ADD: OperationalAxiom enum A7-A13

FÁZE 2 (Týden 2 – Validation Layer):

- └─ EXTEND: verifyIntegrity() pro A7-A13
- └─ ADD: Vitest test suite (axiom compliance + type guard)
- └─ ADD: Pipeline hash chain (MIND1→DEEP_1→DEFENSE)

FÁZE 3 (Týden 3-4 – Python Backend):

- └─ FastAPI microservice (spaCy NLP)
- └─ NLPBridge TypeScript klient
- └─ pytest axiom compliance suite

FÁZE 4 (Ongoing – Control Room Integration):

- └─ Real-time dependency visualization
- └─ Axiom drift detection dashboard
- └─ Pipeline health monitoring

Zdroje

Verifikováno proti SteelsSystem/AI-Forensica live HEAD.

Všechny řádky kódu a čísla ověřeny z klonovaného repozitáře.